

增強課程 9：Key User Extensibility (Custom Fields and Logic / Custom CDS Views / Custom Business Objects)

Lecture

為什麼放在 Enhancement 課程最後：en01 ~ en08 教的東西，一部分在 Cloud 上不存在

en01 ~ en08 教的四大技術 (Classic User-Exit、Classic BAdI、新式 BAdI / Enhancement Spot、Explicit / Implicit Enhancement Point)，全部建立在同一個前提上：**On-Premise** 系統給你完整、不受限制的 **Repository 存取權**——想碰哪個 SAP 物件就碰哪個，只靠 CLAUDE.md「不可修改標準物件」這條團隊紀律自我約束，語言本身完全不檢查。

SAP 近年主推的 **ABAP Cloud** 開發模式，砍掉的正是這個前提：**編譯器強制檢查**你參照的每一個物件是不是被 SAP 明確標記為「Released」(發布契約，見 [ABENRELEASED_API_GLOSRY](#))，沒有標記的物件，程式**編譯不過**，不是執行期才報錯。這條規則對 en01 ~ en08 的四種技術造成完全不對稱的衝擊：

- **Classic User-Exit ([SMOD/CMOD](#))**：這個機制本身假設你能任意編輯 SAP 留的 Include，這個前提在 ABAP Cloud 下不成立——**沒有對應物件型別，整套技術在 Cloud 語言版本下消失。**
- **Explicit / Implicit Enhancement Point (en04 / en07)**：本質是「在別人的原始碼中間插一刀」，需要對著一支具體程式的具體某一行下手。ABAP Cloud 底下，SAP 自己的程式碼不對外開放「插入」這個操作——沒有一個 SAP 物件會被標記成「這裡歡迎你插入 Enhancement Point」，Release Contract 的精神是「只開放**明確定義**的介面，不開放**任意程式碼位置**」——這兩個技術一樣沒有對應語法。
- **BAdI (Classic 與新式 Enhancement Spot、en03 / en05 / en06)**：★唯一活下來、延續進 Cloud 世界的機制——因為 BAdI 從設計之初就是「SAP 明確定義一個 Interface，等你來實作」，天生符合「開放明確介面」的精神。差別只在於：ABAP Cloud 底下你只能實作被明確標記 **Released** 的 BAdI，不能像 en05/en06 那樣 quickSearch 隨便找一個 Enhancement Spot 就用。

一句話總結：**Classic Extensibility** 用「紀律」約束你不要亂改；**ABAP Cloud** 用「編譯器」讓你連改都改不了，而且只留下 **BAdI / CDS View** 這種「明確定義介面」型態的機制活著。

SAP 官方的擴充分類：Key User / Developer / Side-by-Side 三個 Cloud-Ready 選項

SAP 官方文件 ([ABAP Cloud](#) 產品線「Extensibility」頁) 把 Cloud-Ready 的擴充手段分成三類，並且明確標出每一類是 **On-Stack** (在同一套系統裡) 還是 **Side-by-Side** (另開一套系統)：

	Key User Extensibility	Developer Extensibility	Side-by-Side Extensibility
Extensibility Type	On-Stack	On-Stack	Side-by-Side
目標環境	SAP S/4HANA (含 Cloud / Private / On-Premise，同一套系統)	同左 (同一套系統)	SAP BTP，含 BTP ABAP Environment (另一套系統)

	Key User Extensibility	Developer Extensibility	Side-by-Side Extensibility
對象	Key User (不寫程式)	ABAP 開發者	開發者 (可跨語言)
可用物件	BAdI、CDS View (僅限被標成 Key-User 可用 的那個更小子集合)	BAdI、Class、Interface、CDS View、Behavior Definition (僅限 Released 的)	BAPI、IDoc、OData API、SOAP API、Event
語言版本	ABAP for Key Users	ABAP for Cloud Development	ABAP for Cloud Development
工具	Fiori App (精靈 / 簡化編輯器)	ADT (Eclipse)	ADT / 非 ABAP 工具
Benefit	免開發技能，全託管	Lifecycle-stable、可用 Released 物件、 不需要遠端存取與資料複製	解耦、跟主系統 lifecycle 無關

⚠ 容易誤解的地方：On-Stack / Side-by-Side 不是「Developer Extensibility 底下的兩個子分類」，而是一條橫跨三欄的獨立分類軸——**Key User Extensibility 也是 On-Stack** (都在 S/4HANA 系統本身裡面做，同一顆資料庫、同一套 Lifecycle)，只有 Side-by-Side Extensibility 是真正另開一套系統 (BTP)。對「開發者」這個角色而言，程式碼要放在哪，才有「On-Stack vs. Side-by-Side」這個選擇題；Key User 用的是 Fiori App，從來不在這個選擇題裡。

本課只教 **Key User Extensibility** 這一欄——On-Stack Developer Extensibility 需要一套開了 **ABAP Cloud** 語言版本的 **S/4HANA** 系統 (On-Premise 2022+ 或 Private Cloud Edition)，本課程既有的 1909 系統做不到；Side-by-Side Extensibility (真正「另一套系統打 API 回來」的場景) 留給 RAP Cloud 課程的延伸題 (**rc09**，用既有 On-Premise 系統 + **rc** 課程的 BTP ABAP Environment 接起來)。

BAdI 這條線怎麼繼續往「不用寫程式」的方向收斂

上面表格中間兩欄 (Developer / Key User Extensibility)，其實是同一套「Released BAdI / Released CDS View」骨幹，只是包裝方式不同，可以看成一條收斂路線：

世代	誰來實作	工具	範圍限制
Classic BAdI (en03)	ABAP 開發者	SE18/SE19	無——任何 BAdI 都能實作
新式 BAdI / Enhancement Spot (en05/06)	ABAP 開發者	ADT / SE18/SE19	無——本課程 quickSearch 找到就能用
Developer Extensibility (本課程未教，需 ABAP Cloud 系統)	ABAP 開發者	ADT (Eclipse)，限 ABAP Cloud 語言版本	只能實作 Released 的 BAdI / CDS View
Key User Extensibility (本課)	不會寫程式的 Key User	Fiori App (Custom Logic / Custom Fields / Custom CDS)	只能用被進一步標記「Key User 可用」的更小子集合，改用精靈

世代	誰來實作	工具	範圍限制
		Views / Custom Business Objects)	/ 簡化編輯器操作，不開放完整 ABAP / CDS 語法

本課四支 App，各自對應到 en01 ~ en08 學過的哪個機制

- **Custom Logic**：挑一個 SAP 已標記「Key-User 可用」的 BAdI Definition，在簡化編輯器裡寫一小段邏輯——本質跟 en05/06 做的事情是同一件事，差別是不能自己 quickSearch 找任意 BAdI，只能從 App 提供的清單選；也不用開 SE19，用 Fiori 表單填。
- **Custom Fields**：幫一張表加欄位——但只能加在 SAP 明確標記可擴充的表上。這點我們上次查證時在 MARA 的 DDL 裡親眼看到證據：@AbapCatalog.enhancementCategory：#EXTENSIBLE_CHARACTER_NUMERIC 這個 annotation，就是「這張表對 Custom Fields App 開放字元 / 數字型別擴充欄位」的正式宣告——這是資料模型層級的 Release Contract，跟 BAdI 的 Release Contract 是同一種精神。系統會自動用一個保留前綴（本系統實測是 ZZ1_，有些系統設定是 YY1_）幫你的欄位命名，這個前綴不是你自己選的，是系統依 Customizing 決定的命名空間，這點本身也是「Release Contract」精神的體現——連命名都被納入契約管理，避免跟未來 SAP 標準欄位撞名。
- **Custom CDS Views**：組合既有的 Released CDS View，兜出一個新的查詢視圖——概念上跟 CDS 課程 / RAP 課程學過的 EXTEND VIEW（開發者手寫 CDS 擴充語法）是同一件事，只是用精靈點選欄位，不用寫 DDL。
- **Custom Business Objects**：用精靈建一個資料模型 + 自動生成基本 CRUD 畫面——這其實就是 RAP 課程（rap01 ~ rap03）手動教的 Managed RAP BO，被包成一個不用寫 CDS / BDEF / ABAP 的無程式碼精靈。上完 RAP 課程再看這支 App，會很清楚看懂它在背後自動生成了什麼。

Custom Fields and Logic App：三個頁籤的用途和意義

這支 App 名稱裡的「Fields」跟「Logic」對應到 Contract C1 文件講的「Use in Key User Apps」開放的兩種物件型別——Data Elements / Domains（資料模型）與 BAdI Definitions（商業邏輯）——這支 App 把這兩種擴充能力包在同一個入口，外加第三個頁籤處理「怎麼把新欄位真的用到既有畫面 / 報表上」的問題：

- **Custom Fields（幫既有表加欄位）**：只能加在 SAP 明確標記可擴充的表上。上次查證時在 MARA 的 DDL 親眼看到證據：@AbapCatalog.enhancementCategory：#EXTENSIBLE_CHARACTER_NUMERIC 這個 annotation，就是「這張表對 Custom Fields App 開放字元 / 數字型別擴充欄位」的正式宣告——資料模型層級的 Release Contract，跟 BAdI 的 Release Contract 同一種精神。系統會自動用保留前綴（本系統實測是 ZZ1_，有些系統設定是 YY1_）命名新欄位，這個前綴不是你自己選的，是系統依 Customizing 決定的命名空間，連命名都被納入契約管理，避免跟未來 SAP 標準欄位撞名。
- **Data Source Extensions（把欄位/關聯資料接到既有報表 / 畫面上）**：這跟 Custom Fields 是兩件不同的事，容易搞混——Custom Fields 是「幫一張表*新增*一個原本不存在的欄位」；Data Source Extensions 是「把某個既有 CDS View 已經透過 Association 連得到、但沒有曝露出來的欄位（可能來自完全不同的表），額外掛到這個 CDS View 上，讓某支既有的標準報表 / 畫面可以多顯示這個欄位」——概念上等同開發者手寫 CDS association 後在 SELECT 清單多帶一個關聯欄位，只是這裡限定在 SAP 已經定義好、標記可擴充的既有 Association 路徑上，不能自己新建關聯。這個頁籤通常要搭配目標報表/畫面自己的「Adapt UI」功能一起用，才能讓新欄位真的顯示出來；本課的 todoList 沒有對應的既有標準報表可以練這個頁籤，這次先用瀏覽方式了解概念即可，不強求動手。
- **Custom Logic（掛在既有 BAdI 上寫邏輯）**：挑一個 SAP 已標記「Key-User 可用」的 BAdI Definition，在簡化編輯器裡寫一小段邏輯——本質跟 en05/06 做的事情是同一件事，差別是不能自己 quickSearch 找任意 BAdI，只能從 App 提供的清單選；也不用開 SE19，用 Fiori 表單填。這個簡化編輯

器極可能跟 Part 2 用過的「ABAP Fiori 編輯器」是同一套底層元件——官方另一份文件描述的介面是 **Develop / Test / Compare** 三個分頁（寫程式碼、測試不同參數、比較不同版本），跟我們在 Custom Business Object 的 Logic 分頁看到的「Draft/Published 並排 + 內建 Test 面板」是同一種設計哲學的不同呈現。

跟 **Custom CDS Views / Custom Business Objects** 的分工對照：**Custom Fields / Data Source Extensions** 動的是既有 **SAP 標準物件**（加欄位、多顯示欄位）；**Custom CDS Views / Custom Business Objects** 是建**全新**的物件（新查詢視圖、新資料實體）——一個是「擴充別人的」，一個是「蓋自己的」，這個分工跟 en01 ~ 08 教的「不改標準物件，只在旁邊掛外掛」的鐵律精神完全一致，只是換成無程式碼工具做同一件事。

Custom Business Object App 操作細節：Features 核取方塊與 Nodes 頁籤

New 精靈建好空殼後，會進到 **Edit Custom Business Object** 畫面，**General Information** 分頁底下有一個 **Features** 區塊，五個核取方塊決定這個物件之後能用哪些能力（來源：SAP Help「Creating Custom Business Objects」/「Before Getting Started」）：

Checkbox	作用	底層對應（跟 RAP 課程比較）
Determination and Validation	打開之後才能幫這個物件（或它的子節點）加 自訂邏輯 ：Determination / Validation / Action。不勾的話，這個物件永遠只是純資料表，沒有任何客製化業務規則的入口	對應 rap05 （Determination）/ rap06 （Validation）/ rap07 （Action）——差別是不用寫 BDEF 語法，系統自動生成一組「外層 / 內層」Class，實際邏輯寫在內層 Class（用 App 內建的簡化版 ABAP Fiori 編輯器，不是完整 Eclipse ADT）
Service Generation	生成 OData 服務，讓外部（其他程式、Postman、別的系統）可以透過 API 讀寫這個物件的資料。勾選 Generate UI / UI Generation （見下方）會自動連帶勾選這個；也可以只勾這個、不勾 UI ，用在「只要 API、不需要畫面」的情境	對應 rap04 （Service Definition / Service Binding）
Can Be Associated	讓 別的 Custom Business Object 可以透過關聯（Association）指向這個物件，當作參照或 Value Help 用——注意這是「被誰參照」，跟下面 Nodes 頁籤的父子關係是兩件不同的事	概念上類似 RAP 的 Association（不同 Root 之間互相參照）
System Administrative Data	自動生成並自動填值 4 個唯讀欄位： Created On / Created By / Last Changed On / Last Changed By ，完全由系統維護，不能手動改	對應 rap05 手寫的 created_at/created_by Determination——這裡完全不用寫邏輯，勾選就有，是體會「無程式碼版本」最直接的例子
Change Documents	開啟後，這個物件在生成的 UI 上做的每一筆異動（新增/修改哪個欄位、誰改的、何時改的）都會被記錄成 Change Document，可在畫面上查	概念類似 RAP 官方框架的 Change Document 整合，一樣是勾選即有、不用自己寫程式

Checkbox	作用	底層對應 (跟 RAP 課程比較)
	閱。⚠️ 官方文件提醒：資料量大、異動頻繁的物件開這個會影響效能，需要定期用 Information Lifecycle Management 歸檔	

⚠️⚠️ 已更正 (2026-09-06 使用者實測推翻原本的推測)：官方最新文件 ([ABAP_PLATFORM_NEW 2025 FPS01](#)) 的建立流程裡，Features 區塊還有一個 **UI Generation (Generate UI)** 核取方塊，控制要不要自動生成一個可以操作的 Fiori CRUD 畫面 (含 BSP 應用、Semantic Object 等產物)。原本猜測「這個核取方塊要等 Fields 分頁加了 Key 欄位才會出現」——已用實測推翻：加了 Key 欄位 (**ID**) 之後，Features 區塊依然只有五個核取方塊，沒有出現 **UI Generation**。合理推測是這套 1909 系統的 Custom Business Objects App 版本比官方 2025 年文件描述的舊，這個系統版本可能沒有獨立的 **UI Generation** 核取方塊 (UI 生成可能綁在 **Service Generation** 裡自動處理，或完全由 Publish 後系統判斷，尚未查證)。驗證方法：Publish 之後回到 **General Information** 分頁，如果自動生成了 UI，畫面會多出 **Semantic Object 名稱與 SAPUI5 Component ID** (官方文件描述的「已生成 UI」證據)——用這個方法確認比繼續在 Draft 畫面找一個可能不存在的核取方塊更可靠。

這次 **todolist** 練習的建議勾法：**System Administrative Data** (零風險，直接體會無程式碼版本的欄位自動化) + **Service Generation** (之後可以看到自動生成的 OData 服務，對應 **rap04**)；**Determination and Validation** 建議也勾起來，即使不寫邏輯，光是看勾選後多出來的分頁長什麼樣子就有教學價值；**Can Be Associated / Change Documents** 這次練習用不到，可以先不勾 (想實驗的話勾了也不會有風險，只是這個練習不需要)。

Nodes 分頁：定義的是這個物件內部的階層結構，每一列對應底層一張資料表——

- **Root Node**：New 精靈建立空殼時自動產生的那一列，預設跟物件本身同名，一定存在、不能刪除，代表整個物件本身
- 額外用 New 新增的節點：代表 Root Node 底下的子節點 (1 對多的父子關係)，例如官方教學範例的 **Equipment** (Root) 底下加一個 **AssignedTo** (子節點)，代表一台設備可以對應多筆借用記錄

用 RAP 課程對照最清楚：**Nodes** 頁籤做的事情，就是 **rap08** 教的 **Composition (Header-Item 關聯)**——如果 Business Object 之後想改成「一個清單 (Header) 底下有多個項目 (Item)」，就會回到這裡按 New 加子節點，效果等同 **rap08** 手寫的 **composition [0..*] of**。這跟上面 **Can Be Associated** 是兩件不同的事：**Can Be Associated** 是兩個獨立的 **Root** 互相參照 (Association)；**Nodes** 是同一個物件內部長出父子階層 (Composition)。**todolist** 是扁平清單，不需要子節點，Root Node 維持原狀即可，直接去 **Fields** 分頁加欄位。

Fields 分頁：欄位型別與 Properties 面板

選中左邊 **Nodes** 的某個節點後，中間會列出該節點的欄位清單。如果 **General Information** 勾了 **System Administrative Data**，這裡會自動多出 4 筆 **Identifier** 開頭是 **SAP_...** 的欄位 (**Created On / Created By / Last Changed On / Last Changed By**)——官方文件明講這是 SAP 管理的唯讀欄位，不能手動修改，用 **SAP_** 前綴跟你自己建立的欄位做區分。

新增欄位：點 New → 填 Label → 選 Type → 需要當 Key 就勾 Key → 點選那一列選取鈕，右邊 **Properties** 面板設定細節 → Apply。完整 **Type** 清單 (官方文件)：**Amount with Currency / Association to Business**

Object / Checkbox / Code List / Date / Email Address / Number / Numeric Identifier / Phone Number / Quantity with Unit / Text / Time / Timestamp / Web Address。

⚠ 自己建立的欄位 **Identifier** 不需要、也不應該加 **Z/Y** 前綴：**Z/Y** 命名空間規範的是獨立的 **Repository** 物件 (Class、Program、Table、Function Module)，這裡建的 **ID / DESCRIPTION** 這類是這張表內部的欄位 (**Column**)，性質上跟標準表 **MARA** 底下的 **MATNR / WERKS** 欄位一樣——命名空間隔離已經在物件層級做掉了 (這個 Business Object 本身的技術名稱系統會強制加保留前綴，如 **ZZ1_TODOLIST**)，欄位名稱只要在這張表內部不重複、清楚易懂即可。⚠ 中文 **Label** 沒辦法自動推導出合法的技術名稱：英文 Label (如 **ID**) 系統會直接拿來當 Identifier，但中文 Label (如「是否完成」) 系統推導不出來，會給一個無意義的通用序號佔位符 (如 **Field007**)，這種情況要手動把 Identifier 改成有意義的名稱 (如 **IS_COMPLETED**)，建議趁還沒 Publish、還能自由修改時就處理好。

Properties 面板的六個欄位，依選的 **Type** 不同而決定哪幾格用得到：

Properties 欄位	意義	適用的 Type
Tooltip	滑鼠停在欄位上顯示的提示文字	所有型別，選填
Length	文字長度上限	只對 Text 有意義
Decimals	小數位數	只對 Number / Amount with Currency / Quantity with Unit 有意義
Code List	綁定一份預先在 Custom Reusable Elements App 建好的值清單，變成下拉選單	只有 Type=Code List 才用到
Business Object	指定這個欄位要關聯到哪一個其他 Custom Business Object——這是「Association」關係實際設定的地方	只有 Type=Association to Business Object 才用到
Read Only	手動把欄位設成唯讀。System Administrative Data 生成的欄位這裡會強制勾選、無法取消	所有型別

呼應上面的 **Can Be Associated**：**Can Be Associated** 是「允許自己被別人指」(在被參照的物件上開)，**Association to Business Object** 型別 + **Business Object** Property 才是「自己去指別人」的實際操作位置 (在發起參照的物件上，建欄位時指定)——兩者合起來才是完整的雙向關聯設定。

這次練習三個欄位的建議設定：**ID** (**Text**，勾 **Key**，**Length** 設 10) / **描述** (**Text**，**Length** 設 100) / **是否完成** (**Checkbox**，不需要 **Length/Decimals**)。

⚠ 官方限制：欄位一旦 Publish 後走過傳輸，以下改動不再允許：縮短 **Length**、拿掉 **Key** 標記、更換 **Code List** 或 **Association** 目標、更改數值型別的 **Decimals**、把 **Date/Time** 換成其他型別——跟 Custom Fields 「Published 後不可逆」是同一種設計精神。

Logic 分頁：Draft 只能宣告，實作要等 Publish 之後

打開 **Logic** 分頁，即使 **Determination and Validation** 已勾選，畫面會顯示提示訊息：「**You can only access logic implementation in Published mode.**」——這代表 **Draft** 階段只能先「宣告」要有哪些 **Action / Determination / Validation** (用 **New** 建立 Label / Identifier，決定叫什麼名字、掛在哪個節點)，實際寫程

式邏輯要等 **Publish** 之後才能做，呼應官方文件說的「You can implement action logic **after** you have published the custom business object」。這跟前面 **Custom Logic App**（掛在標準 BAdI 上）的運作邏輯很像：先建立空殼 / 宣告，再回頭補實際邏輯，只是這裡的「空殼」是整個 Business Object 要先 Publish 過一次。

✅ 已用 **Publish** 驗證確認：Draft 階段 Logic 分頁確實只看得到 **Actions of TODOLIST**；**Publish** 之後，分頁多出一個 **Determination and Validation** 區塊，裡面是兩個系統內建、固定命名的掛勾點（不是自己命名的清單）：**After Modification**（型別 Determination，欄位被修改時觸發）/ **Before Save**（型別 Validation，存檔前觸發）——跟官方文件「Adding Logic to Actions, After Modification, and Before Save」描述的行為一致。跟 **RAP** 課程對照：**rap05/rap06** 可以自訂任意數量、任意命名、綁定特定欄位或特定 CUD 操作的 Determination / Validation；這支 App 只給兩個固定通用掛勾點——這是精靈犧牲彈性換取簡單的具體例子。

另一個實測發現：**Fields** 分頁編輯完欄位後，左下角的驗證訊息小方塊（曾經顯示「Node TODOLIST requires at least one user-defined field.」）在按過 **Save Draft** 之後就消失了——代表這類驗證訊息不會即時跟著你的編輯動作重新整理，遇到訊息內容跟畫面實際狀態對不上時，先 **Save Draft** 觸發重新檢查，而不是急著懷疑自己漏做了什麼。

Publish 的完整 **Log** 步驟（**Log** 分頁）：**Generation of runtime object names** → **data elements** → **tables** → **CDS views** → **key** → **business logic** → **business object runtime** → **classification** → **service** → **change document object** → **Final checks** → **Success**。這幾乎逐字對應 **RAP** 課程手動教的步驟：Data Element / Table（DDIC）→ CDS View → Key → Business Logic（等同 BDEF）→ Business Object Runtime（等同 Behavior Pool / Service 執行期），是這一課「精靈自動做掉 RAP 手動步驟」這個論點最直接的證據。

⚠️ 確認：這個系統版本 **Publish** 後沒有出現 **Semantic Object / SAPUI5 Component**——**General Information** 只多出 **Name of Technical Service**（如 **ZZ1_TODOLIST_CDS**）/ **Version of Technical Service**，代表 **Service Generation** 確實生成了 OData 服務，但沒有生成 **Fiori UI**。這印證了前面「這個系統版本可能沒有 UI Generation 能力」的推測。

寫 Determination / Validation / Action 邏輯：語法與內建測試工具

點進 **After Modification**（或 **Before Save** / 某個 Action），會看到 **Published Logic**（唯讀骨架，用註解列出可用參數）——要編輯先點 **Create Draft**，才會出現可編輯的 **Draft Logic** 區塊，寫完點 **Publish** 生效。三種 Importing/Changing 參數（官方文件「Adding Logic to Actions, After Modification, and Before Save」確認）：

參數	類型	用途
association	Importing	導覽到父 / 子 / 關聯節點的實例（用 association->to_<關聯名>（）這種語法），單一扁平節點用不到
write	Importing	寫入其他 Custom Business Object 的 API，這次練習用不到
<節點名>（如 TODOLIST）	Changing	當下這一筆資料的結構，直接用 <節點名>-<欄位> 讀寫，這是 Determination / Validation 主要操作對象

參數	類型	用途
<code>message</code> (Actions / Validation 都有，寫法不同)	Exporting	Actions 用 <code>message = VALUE #(severity = co_severity-warning text = '...')</code> 。(<code>co_severity</code> 有 <code>success/warning/error</code> 等級) ; Validation (Before Save) 用純字串： <code>message = '...'</code> 。
<code>valid</code> (僅 Validation / Before Save 有)	Exporting	<code>abap_true/abap_false</code> ——真正決定要不要擋下存檔的開關，設 <code>abap_false</code> 才會真的擋住存檔動作，光設 <code>message</code> 不設 <code>valid</code> 不會擋

範例① (**After Modification**，把描述自動轉大寫)：

```
TODOLIST-DESCRIPTION = to_upper( TODOLIST-DESCRIPTION ).
```

範例② (**Before Save**，描述空白時擋下存檔——官方文件「Adding Logic to Custom Business Objects」**Before Save** 標準範例套用到本例欄位)：

```
IF TODOLIST-DESCRIPTION IS INITIAL.
    message = 'Please enter a description.'.
    valid = abap_false.
ELSE.
    valid = abap_true.
ENDIF.
```

跟 **RAP** 課程對照：這個 `valid/message` 組合，本質上就是 **rap06** 教的 `failed/reported` 表格 (Validation 失敗回報錯誤) 的無程式碼簡化版——不用組 `%key / %msg` 結構，直接設兩個平面變數即可。

✅ 已用 **Test** 面板實測驗證成功，過程中發現兩個值得記住的細節：

1. **VALID** 欄位顯示「空白」代表 `abap_false`，不是顯示 `false` 這幾個字——ABAP Boolean 底層 `abap_true='X'、abap_false=' '` (空白字元)，Test 結果面板忠實呈現這個底層值，第一次看到空白框容易誤以為「沒有值」或「還沒執行」，其實正是 `abap_false` 的正確顯示方式
2. **Before Save** 的 **Published** 骨架，預設就帶了 `valid = abap_true`。這行可執行程式碼——跟 **After Modification** 的骨架 (只有註解、沒有任何程式碼) 不一樣。這是合理的安全設計：Validation 決定「能不能存檔」，系統要有個預設放行的安全值，避免使用者自訂規則之前所有存檔都被誤擋

實測兩組對照 (`description` 空白 vs. 有值)：空白時 **Draft Logic** 回傳 **VALID**=空白 (`abap_false`) + **MESSAGE**='Please enter a description.'；有值時兩邊都是 **VALID**='X' (`abap_true`)、**MESSAGE** 空白——邏輯正確區分兩種情境，不會誤擋正常資料。

驗證方式不需要真的建 UI 或用 **Gateway Client**：編輯畫面內建測試工具，✅ 已實測確認完整操作流程：

1. 畫面上方 <節點名>：(如 **TODOLIST**) 欄位右邊有一個小圖示——點它會彈出 **Maintain Node Data**：<節點名> 對話框，裡面是 **Field / Value / Type** 表格，每個欄位都是真正可輸入的文字框，在這裡填測試值 (例如 `id` 填 `001`、`description` 填小寫 `test`)，填完點 **OK**
2. 點下方工具列的 **Test** 按鈕

3. 畫面下方會展開兩個並排區塊：**Test Results: Draft Logic / Test Results: Published Logic**——同一組測試輸入分別跑 **Draft** 版本與目前已 **Publish** 版本的邏輯，方便直接對照兩者行為差異

實測結果：輸入 `description = 'test'`，跑完 **Test** 後 **Draft Logic** 結果顯示 `description = 'TEST'`——證實 `to_upper()` 正確執行；**Published Logic** 那邊維持空白（因為 **Published** 版本目前還是空骨架，什麼都不做），兩邊對照完全符合預期，這代表 **Determination** 邏輯已經完整驗證成功，不需要真的建 UI 或用 **SAP Gateway Client** 才能確認。

⚠ 這是共用 Demo 系統，操作前務必記住

Custom Fields and Logic → **Custom Fields** 頁籤目前已有 6 筆其他人建立、大多已 **Published** 的真實欄位（**ZZ1_CustomFieldHighRis / ZZ1_CustomFieldRiskMit / ZZ1_CustomFieldRiskRea / ZZ1_MARC1 / ZZ1_SAPCODE_MARC1 / ZZ1_ZZSAPCODE**，Business Context 都在 Product 相關）——這些不是課程教材，是真實客製化，不要點開、修改或刪除。**Published** 狀態的 Custom Field 一旦建立會真的在資料庫加欄位（DDIC 結構異動），不像一般 `$TMP` Z 物件可以隨便重建；本課設計為只建立到 **Draft / Not Published** 狀態（畫面上 **ZZ1_MARC1** 目前就是 **Not Published**，證實這個中間狀態是被允許、可以安全停留的），不會實際變更資料庫結構。

學習目標

- 能講出 Classic Extensibility（en01 ~ 08）跟 Key User Extensibility 在「碰的到什麼物件」「誰來操作」「用什麼工具」三個面向的差異
- 能講出 Explicit / Implicit Enhancement Point 為什麼在 ABAP Cloud 下沒有對應語法，而 BAdI 為什麼是唯一延續下去的機制
- 能講出 On-Stack Extensibility 與 Side-by-Side Extensibility 的差異，並指出 Key User Extensibility 跟 Developer Extensibility 都屬於 On-Stack
- 能操作 Custom Business Objects App，從頭建立一個簡單物件，並說出這個精靈幫你自動做掉了 RAP 課程裡哪些手動步驟
- 能操作 Custom Fields and Logic App 建立一個欄位到 Draft 狀態，說出這個流程對應 en05/06 教的 DDIC Data Element / Domain 建立的哪些概念
- 能講出 Custom CDS Views App 的 **Access Protection** 欄位（**Protected / None**）可能代表的意義，並連結到 Release Contract 的概念

事前準備

- 環境：On-Premise S/4HANA 1909，Client 130，Fiori Launchpad 網址 <https://erpdemo01.itts.com.tw:44300/sap/bc/ui2/flp>（用你既有帳號登入，跟 RAP / CDS / Enhancement 課程共用同一套系統）
- 這一課跟前面八課的驗收方式不一樣：這四支 App 沒有已知的 ADT / MCP 讀寫介面（見 [.claude/rules/sap-adt-mcp.md](#) 第 59 節），Claude 沒辦法像其他題目一樣「先實測產生範例答案」——本題流程是你操作 → 截圖回報 → **Claude** 核對與講解，跟 Smartform 課程、CMOD 專案指派步驟的驗收方式相同
- 開始前先用 App Finder 搜尋確認這三支 App 都找得到：**Custom Fields and Logic / Custom CDS Views / Custom Business Objects**（上一輪對話已經確認過都在）

題目需求

第一部分：概念整理（不動系統）

1. 完成一張表格：欄位是「技術名稱 / 年代 / 誰刻意留插入點 / 能不能碰任意物件 / 在 ABAP Cloud 下還能不能用」，列出 en01 ~ en08 的四大分類 + 本課的 Key User Extensibility，一共五列
2. 用一句話解釋：為什麼 Explicit / Implicit Enhancement Point 在 ABAP Cloud 下完全沒有對應語法，而 BAdI 卻可以延續下去

第二部分：Custom Business Objects——動手建立（低風險，目前 0 筆，完全可逆）

1. 開啟 **Custom Business Objects App**，用 **New** 精靈建立一個簡單物件（例如「待辦清單」，2~3 個欄位：**ID / 描述 / 是否完成**），走完精靈到能存檔 / Publish 為止——**Features** 核取方塊要勾哪些、**Nodes** 分頁要不要動，照上面 **Lecture「Custom Business Object App 操作細節」**那一節的建議做
2. 截圖回報精靈的每一步畫面，Claude 會逐步對照：這個精靈的哪一步，對應 RAP 課程 **rap01**（建 CDS View）/ **rap02**（Managed BDEF）/ **rap04**（Service Definition / Binding）的哪個手動步驟
3. 寫下你的觀察：這個精靈幫你自動做掉了哪些 RAP 課程要手寫的東西？又犧牲了哪些 RAP 課程教過的彈性（例如 Determination / Validation / Action 這類自訂邏輯，Custom Business Object 精靈能不能做到）？

第三部分：Custom Fields and Logic——動手建立（只做到 Draft，不 Publish）

1. 開啟 **Custom Fields and Logic** → **Custom Fields** 頁籤，點 **+** / **Create** 建立一個新欄位，Business Context 挑一個跟現有 **6 筆不衝突的**（例如換一個模組的 Master Data Context，或先跟系統管理者確認安全的 Context），型態隨意（Text / Checkbox 皆可）
2. **⚠ 存到 Draft / Not Published 狀態就停手，不要按 Publish**——截圖回報欄位屬性設定畫面（Label / Identifier / Business Context / Type 這幾欄）
3. 觀察系統自動產生的 Identifier（欄位技術名稱）用了什麼前綴，寫下你的猜測：這個前綴是誰決定的、為什麼不讓你自己輸入完整技術名稱
4. 點開 **Custom Logic** 頁籤，截圖回報這個系統的 Business Context 清單裡，有沒有任何一個提供「Custom Logic」（掛 BAdI）的選項；如果有，找一個看它綁定的是哪個 BAdI / Business Context，對照 en03 / en05 / en06 學過的機制說明它的定位；如果沒有找到任何可用項目，也記錄下來並說明可能原因（提示：不是每個 BAdI 都會被 SAP 標記成 Key-User 可用，這本身就是一個有意義的觀察）

第四部分：Custom CDS Views——唯讀探索

1. 開啟 **Custom CDS Views App**，用 Search 找 3 個你熟悉模組（如 MM / SD / PP）相關的 Data Source（如 **C_ApplicationDocStorDets** 這類），記錄它們的 **Access Protection** 欄位值（**Protected / None**）
2. 查 SAP 官方文件裡 Release Contract（C0 / C1）的定義，寫一段話推論：**Access Protection** 欄位的值，會不會就是這個 CDS View 的 Release Contract 狀態的另一種呈現方式？

參考答案

這一課沒有 ABAP 程式碼形式的參考答案——四支 App 都是 GUI 操作，正確與否要靠截圖核對。Claude 會在你回報每一步截圖後，即時對照上面 Lecture 提到的概念逐項確認，並指出跟 en01 ~ en08 哪個機制對應。

思考題

1. Custom Business Object 精靈生成的底層物件，如果之後想要的邏輯超出精靈能做的範圍（例如需要 Determination / Validation），你覺得下一步該往哪裡走？（提示：想一想 Developer Extensibility 那一欄的定位，跟 Key User Extensibility 是不是設計成「接得上」的兩個階段）

2. 這一課示範的 Custom Fields，只能加在系統標記 #EXTENSIBLE_CHARACTER_NUMERIC 這類的表上。如果有一張表完全沒有標記任何 enhancementCategory，你覺得 Custom Fields App 建立欄位時會發生什麼事？有沒有辦法自己動手驗證這個猜測（不用真的 Publish，用 App 的即時檢核訊息就能觀察）？
3. 這一課學到「Key User Extensibility 也是 On-Stack」——想一想：如果你是系統管理者，會希望哪些人有權限用 Custom Fields and Logic App？這跟 en01 ~ en08 那些技術「只有 ABAP 開發者能碰」比起來，對企業的授權管理（誰能改系統行為）帶來了什麼本質上的變化？